

Signal Project Group 22

Parham Moshashaei, Mohammad Mosayebzadeh, Mani Gholampour

402111389, 402110854, 402111378

Abstract

This project presents the design and implementation of an integrated object detection and tracking system for video analysis, developed as a final project for the Signals and Systems course at Sharif University of Technology. The system is engineered to first identify a target object within a video stream using a pre-trained deep learning model and subsequently track its movement across successive frames with a custom-developed tracking algorithm.

The initial detection phase can leverage established models like YOLO or Faster R-CNN to accurately localize the object of interest in the initial frames. Following this, the core of the project is a novel tracking algorithm inspired by state-of-the-art methods. For comparative analysis, three prominent tracking algorithms—CSRT, MOSSE, and KCF—were also implemented.

The proprietary tracking algorithm integrates several key features to ensure robust performance. It establishes temporal dependency between video frames and dynamically adjusts the bounding box size as the object moves closer to or farther from the camera. Temporary occlusions or instances where the object leaves the field of view were handled. Furthermore, to achieve high processing speeds suitable for real-time application.

The final system was evaluated on several criteria, including processing speed (Frames Per Second), the ability to maintain tracking during occlusions, adaptability to changes in the object's appearance and scale, and performance under varying lighting conditions. The output is a video stream that displays the live tracking results, including the bounding box around the target and the real-time FPS metric. The project was developed using Python, with version control managed via [Github](#) as required.

Contents

		4.2 Precision	5
		4.3 FPS	5
1	YOLO	2	
2	CSRT	3	
	2.1 Implementation	3	
	2.2 Precision	3	
	2.3 FPS	3	
3	Mosse	4	
	3.1 Implementation	4	
	3.2 Precision	4	
	3.3 FPS	4	
4	KCF	5	
	4.1 Implementation	5	
		5 The designed algorithm	6
		5.1 Mohammad's algorithm	6
		5.1.1 Implementation	6
		5.1.2 Precision	6
		5.1.3 FPS	6
		5.2 Mani's algorithm	7
		5.2.1 Implementation	7
		5.2.2 Precision	7
		5.2.3 FPS	7
		5.3 Parham's algorithm	8
		5.3.1 Implementation	8
		5.3.2 Precision	9
		5.3.3 FPS	9

1 YOLO

YOLO, which stands for You Only Look Once, is a highly efficient object detection algorithm prized for its exceptional speed. Its core innovation lies in treating object detection as a single, unified regression problem instead of a multi-stage task. The network looks at the entire image just one time, dividing it into a grid and simultaneously predicting bounding boxes, confidence scores, and class probabilities for each grid cell. This single-pass architecture is what makes YOLO fast enough for real-time applications like live video analysis.

The primary reason for choosing YOLO over an older architecture like R-CNN (Region-based Convolutional Neural Network) is its significant advantage in speed and simplicity. R-CNN and its successors are two-stage detectors; they first propose numerous potential "regions of interest" in an image and then run a classifier on each of these regions separately. This two-step pipeline is inherently slow and computationally intensive. In contrast, YOLO's single-stage approach performs both localization and classification in one go, making it vastly faster and more suitable for projects where real-time performance is a critical requirement.

The provided Python script demonstrates a complete and practical application of this model. It begins by downloading both a pre-trained YOLO model and a sample image from the internet, storing them in a temporary location. The script then loads this model into memory and uses it to perform object detection on the downloaded image. To conclude the process and present the findings, it iterates through the detection results, draws green bounding boxes around each identified object directly onto the image, and then displays this final annotated image to the user.



Figure 1: YOLO was used to detect this dog

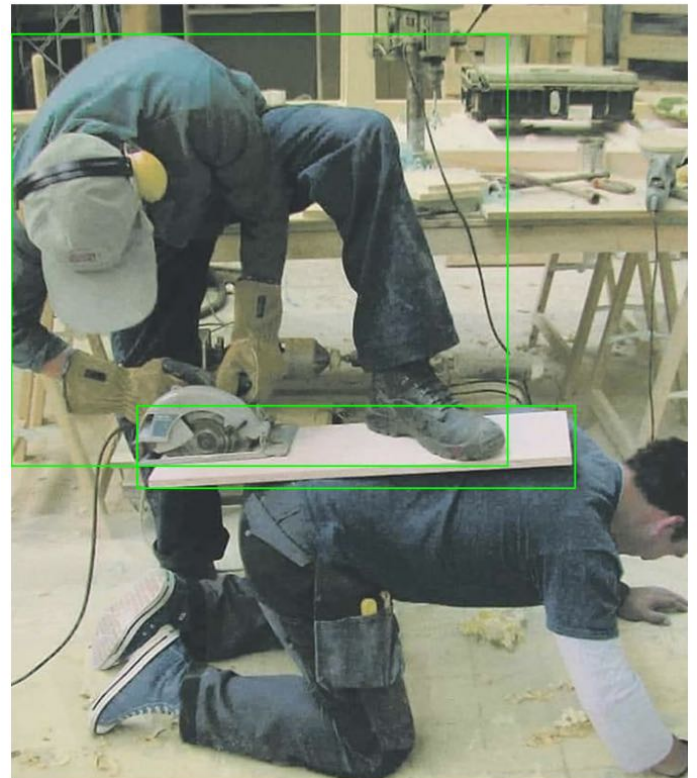


Figure 2: YOLO being used on a more complicated picture

2 CSRT

2.1 Implementation

The CSRT (Channel and Spatial Reliability Tracker), a robust algorithm that follows a specific object's movement throughout a video. Unlike object detection, which identifies objects from scratch in every frame, this script focuses on tracking. It requires the user to initialize the process by identifying a target object once, which the program then follows continuously. This makes it efficient for tasks where you need to monitor a single object's path or behavior over time.

The program's execution is a guided, interactive process. It begins by prompting you to select a video source, offering a choice between your live laptop camera or a pre-existing video file on your computer. After you make your selection, the script displays the first frame and pauses, allowing you to use your mouse to draw a bounding box, also known as a Region of Interest (ROI), around the specific object you want to track. The script includes helpful error checks to ensure a valid video source is provided and that the selected area is large enough to be tracked effectively.

Once the object is selected, the CSRT tracker is initialized and the main tracking loop begins. The script processes the video frame by frame, and in each new frame, the tracker's update function predicts the new position of the target. To provide visual feedback, a blue rectangle is drawn around the object's current location in real-time. The display is enhanced with useful information, such as the tracking speed in Frames Per Second (FPS) and the current status. For video files, the script will automatically loop back to the beginning when it's finished, allowing for continuous observation. You can stop the entire process at any time by simply pressing the ESC key.



Figure 3: CSRT selection on person2.mp4

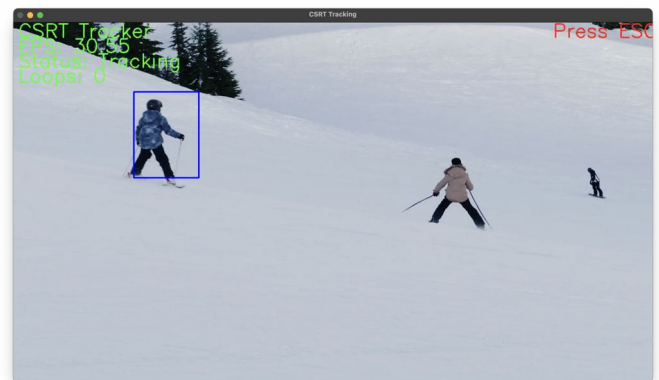


Figure 4: CSRT showing output and fps on person2.mp4

2.2 Precision

Based on the output footage and looking at the other algorithms in my view CSRT has the best precision when it comes to tracking as shown when comparing 3 and 4

2.3 FPS

as shown in 4 the fps it reaches is around 30 fps on the person2.mp4 file which is a good and natural feeling fps however comparing to the other two feels lacking.

3 Mosse

3.1 Implementation

The MOSSE (Minimum Output Sum of Squared Error) object tracker. In this implementation the cv2 version wasn't used and it was implemented by hand. MOSSE is a foundational algorithm in the family of high-speed correlation filter-based trackers. Its defining characteristic is its exceptional speed, which is achieved by performing the core tracking operations in the frequency domain using the Fast Fourier Transform (FFT). The script packages this algorithm into a `MOSSETracker` class and provides a user-friendly interface to select a video source, choose an object, and watch the tracking in real-time, making it a powerful demonstration of how classical, high-performance trackers work under the hood.

The tracking process begins with an interactive initialization step. After the user selects a video source, the first frame is displayed, and the program pauses, allowing the user to draw a bounding box around the target object. Once selected, the tracker's initialize method creates a special template called a correlation filter. This filter is mathematically optimized to produce a sharp peak (an ideal Gaussian response) when it perfectly aligns with the target object's appearance. To improve its initial robustness, the script also trains this filter on several slightly rotated and scaled versions of the object, preparing it for minor variations in the video.

With the filter ready, the script enters its main tracking loop. For each new video frame, the tracker uses the learned filter to find the object's new location. This is done by efficiently cross-correlating the filter with a patch from the new frame in the frequency domain. The spot on the resulting response map with the highest value, or peak, indicates the object's new position. The script then updates the object's coordinates and immediately uses this new view of the object to refine the filter, governed by a learning rate. This allows the tracker to constantly adapt to gradual changes in the object's appearance. Finally, a bounding box

is drawn at the new location, and the frame is displayed with performance statistics like the current and average FPS.

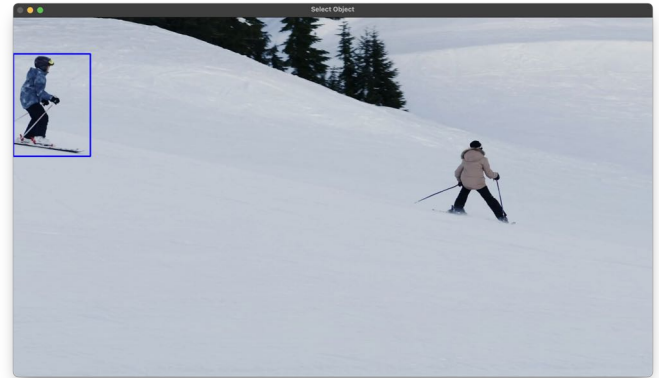


Figure 5: MOSSE selection

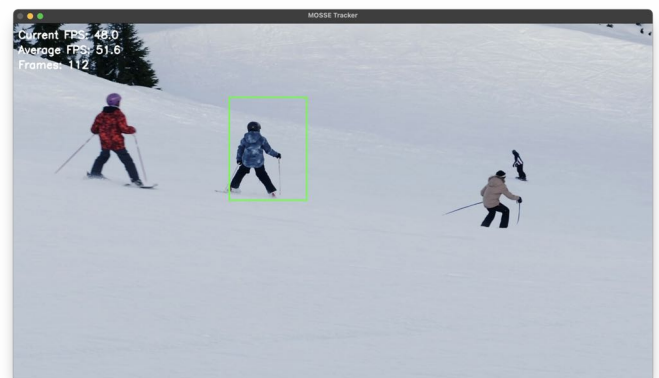


Figure 6: MOSSE showing output and fps on person2.mp4

3.2 Precision

In terms of precision it is less precise than CSRT but it the same level as KCF and this can be felt when looking at the output 6 vs its 5.

3.3 FPS

Looking at 6 the average FPS is around 50 which is the best one in the three we are comparing.

4 KCF

4.1 Implementation

The KCF (Kernelized Correlation Filter) tracker. Like the MOSSE tracker, KCF is a type of high-speed correlation filter, but it improves upon the basic concept by using the "kernel trick." This mathematical technique allows it to create a more sophisticated and non-linear model of the object, making it more robust to changes in appearance and illumination. KCF represents a fantastic middle ground in tracking, offering significantly better performance and reliability than a basic tracker like MOSSE, while still being much faster than more computationally intensive trackers like CSRT.

The program's workflow is guided by clear user instructions. It begins by prompting you to choose between a live webcam feed or a local video file as the source. Once the video loads, the first frame is displayed with an instruction to press the SPACE key. Hitting SPACE activates OpenCV's Region of Interest (ROI) selection tool, allowing you to click and drag your mouse to draw a bounding box around the object you want to follow. After you've defined the target, the KCF tracker is initialized with this selection, and the real-time tracking process begins.

Once initialized, the script enters a continuous loop, processing the video frame by frame. In each frame, the `tracker.update()` method is called to predict the object's new location. If the tracking is successful, a green rectangle is drawn around the object's updated position. If the tracker loses sight of the object, the script provides immediate feedback by displaying a "Tracking failure" message on the screen. To give a sense of performance, the application also calculates a smoothed average Frames Per Second (FPS) rate from recent frames and overlays it on the video, allowing you to monitor the tracker's speed as it runs. The program continues until you press the ESC key to exit.

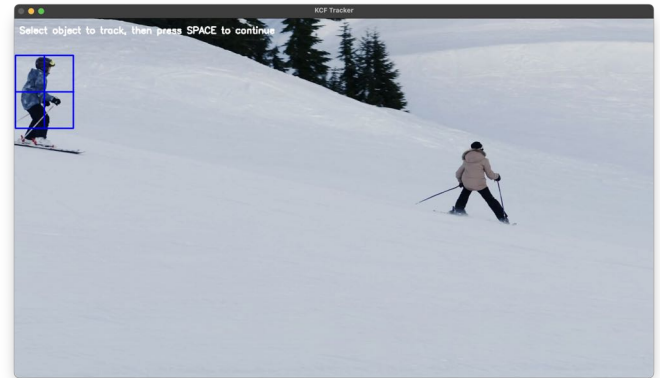


Figure 7: KCF selection

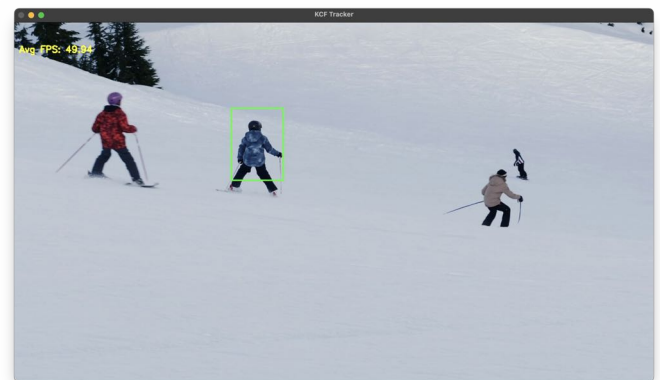


Figure 8: KCF showing output and fps on person2.mp4

4.2 Precision

Whilst better than MOSSE it has some flickering and a little instability when compared to CSRT which is much more persent in film. 8 against 7 tells us the squere remains mostly the same.

4.3 FPS

Looking at 8 the FPS however around 50 which is around the same performance of MOSSE and much better than CSRT.

5 The designed algorithm

5.1 Mohammad's algorithm

5.1.1 Implementation

Its core strategy is designed to maximize performance by not running the computationally expensive YOLO detector on every single video frame. Instead, it performs a full detection only at periodic intervals. In the frames between these detections, it simply carries over the locations of the previously found objects, creating a "tracking" effect. This intelligent trade-off between constant accuracy and speed results in a much higher Frames Per Second (FPS) rate, making the approach highly practical for real-time applications.

The central logic is encapsulated within the YOLO-MultiTracker class, which is initialized with a path to a YOLO model, a confidence threshold, and a specific detection-interval. The threshold ensures that only objects detected with a certain level of certainty are considered, reducing visual noise from false positives. The update method is the key to the script's efficiency; it checks if the time elapsed since the last detection has surpassed the set interval. If it has, the method runs a new detection on the current frame to get a fresh list of objects. If not, it instantly returns the list of objects from the last successful detection, skipping the heavy computation entirely.

From the user's perspective, the application is straightforward. After selecting a video source, a window appears displaying the video stream with all detected objects highlighted. Green bounding boxes are drawn around each object, complete with a label showing its class name and confidence score. To make its internal process clear, the script displays a status on screen: "Detecting" appears during the frames when the YOLO model is actively running, while "Tracking" is shown on the intermediate frames that reuse old data. This, along with real-time FPS counters, provides excellent visual feedback on both the tracker's findings and its

performance until the user presses the ESC key to exit.

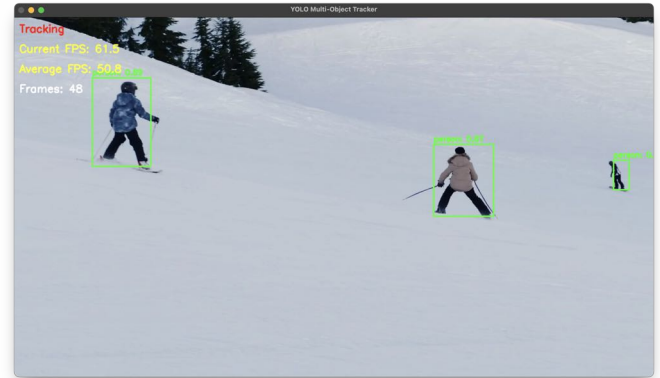


Figure 9: The output of this algorithm

5.1.2 Precision

The Precision is acceptable as it tracks each person uniquely and follows them and even when two people cross roads it follows them correctly comparing 9 and 10.



Figure 10: Two people crossing

5.1.3 FPS

Looking at 10 and 9 FPS hovers around 50 and 60 depending on how many objects it is tracking.

5.2 Mani's algorithm

5.2.1 Implementation

It uses a pre-trained YOLOv5 model to perform periodic, high-accuracy detection of a person in the video. For the frames in between these detections, it employs a custom tracker that fuses a Kalman Filter with Optical Flow. This "detect-then-track" strategy leverages the raw accuracy of YOLO to find and periodically re-acquire the target, while the more lightweight classical methods handle the smooth frame-to-frame tracking, creating a system that is both precise and computationally efficient.

The core of the frame-to-frame tracking logic resides in the custom tracker, which maintains a continuous track of the object between the periodic YOLO detections. It uses two key tools working in tandem. The Kalman Filter serves as the predictive brain of the operation, constantly estimating the object's future position and size based on its past motion, which provides a smooth trajectory. This prediction is then refined by the Optical Flow algorithm, which acts as the system's eyes by tracking the movement of distinct visual feature points (like corners) on the object's surface. This real-world visual measurement is used to correct the Kalman Filter's mathematical prediction, making the tracker highly resilient. If the visual tracking temporarily fails (e.g., due to blur or partial occlusion), the system can gracefully fall back on the Kalman prediction alone.

In practice, the system begins by using YOLOv5 to detect the largest person in the initial frame, initializing the custom tracker on them. For a set number of frames, this custom tracker takes over, using its combined Kalman and Optical Flow logic to follow the target smoothly. After this interval, the powerful YOLOv5 detector runs again, and its new high-confidence detection is used as a "ground truth" to correct any drift the tracker may have accumulated. This cycle of detect, track, track, ..., re-detect ensures the system remains locked onto the correct target with high precision while maintain-

ing fluid, efficient tracking. The bounding box even changes color on-screen to visually indicate whether it's relying on a fresh detection or its own internal tracking.

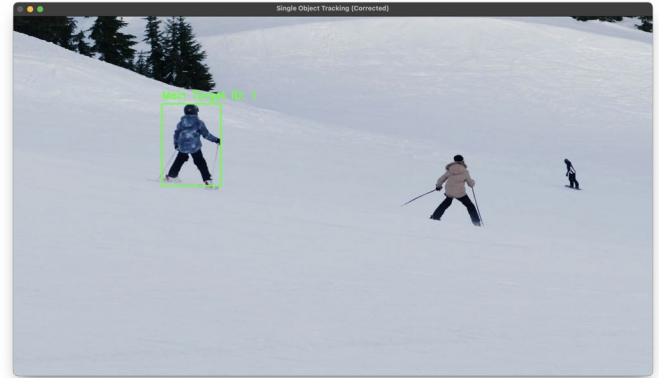


Figure 11: Mani's tracker

5.2.2 Precision

Because it is a single object tracker its precision is depending on how many objects are on the screen and which one the yolo algorithm decides to track. In single tracking scenarios like person1 11 it is amazing but lacks in scenarios like person2 11.

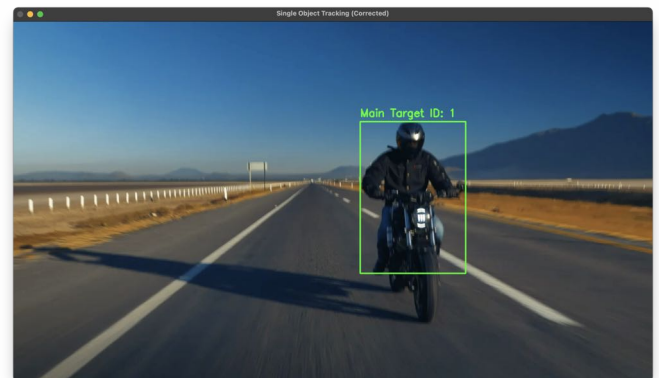


Figure 12: Mani's on person1

5.2.3 FPS

It gives a FPS reading of around 30.

5.3 Parham's algorithm

5.3.1 Implementation

This algorithm was inspired by mills algorithm and in many ways reflects it. The architecture is built on two main pillars: a foundational tracker that learns an object's appearance on the fly, and a high-level YOLOTracker that uses a YOLOv11 deep learning model to supervise and correct the classical tracker. This hybrid design leverages the raw speed of the classical tracker for smooth frame-to-frame motion while using the superior accuracy of the YOLO model to prevent long-term drift and re-acquire targets, creating a system that is both fast and resilient.

At the heart of the system is an incredibly efficient tracker that learns to recognize an object by asking thousands of simple questions about its visual patterns. Using a clever pre-calculation trick on the image called an integral image, it can instantly analyze differences in light and dark across the object to detect key features like edges, lines, and textures. But the real genius is in its adaptive learning engine. Instead of treating all these visual checks equally, it intelligently determines which specific patterns are most unique to your target. It builds a highly accurate final verdict by combining the answers from the most relevant checks, effectively learning a visual fingerprint for the object. This allows it to stay locked on with incredible speed while its on-line learning capability lets it adapt as the object's appearance slowly changes.

The YOLOTracker class acts as the intelligent supervisor, orchestrating the entire process and enabling advanced capabilities like multi-object tracking. It periodically runs the powerful YOLO model to get a high-confidence, full-frame detection of all objects. These detections are then used as a "ground truth" to correct the position of the classical tracker(s), effectively eliminating drift. In its multi-object mode, the system can manage numerous independent trackers at once. It uses an intelligent association algorithm to match the new YOLO detections to the existing tracks, ensuring object identities are maintained over time. This allows the system to seamlessly handle objects entering or leaving the scene by automatically spawning new trackers and retiring old ones.

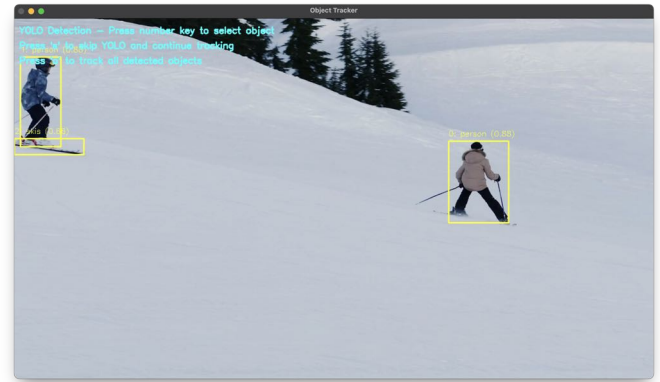


Figure 13: The input screen allowing you to choose what you want

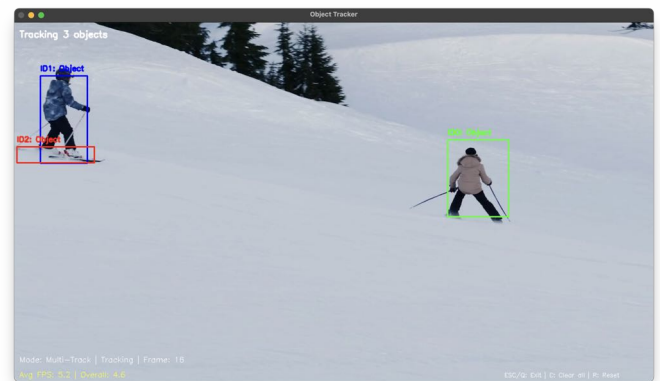


Figure 14: Multi object tracking

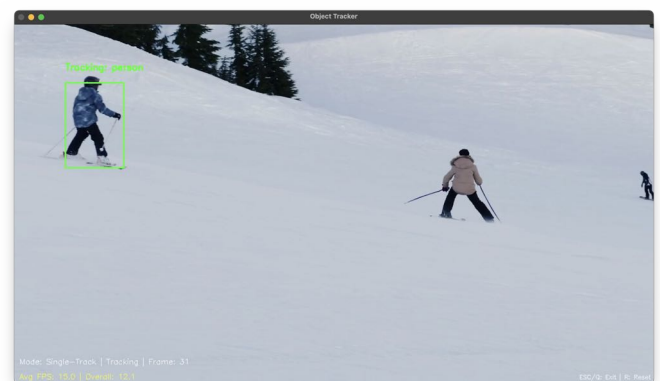


Figure 15: Single object tracking

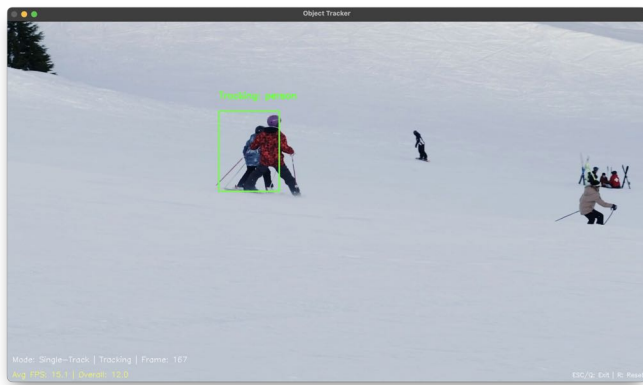


Figure 16: Tracking a object when it is behind another object

5.3.2 Precision

It is very precise because how the tracking and yollo are combined and in single tracking mode it even tracks when the person is behind another completely 16 vs 15.

5.3.3 FPS

Looking at 14 in multi object tracking its fps is around 5 fps and in single object tracking in 15 the fps is around 15 which is more lacking that the other models but it is very very precise.